

No Engine Required: Teaching Game Programming (Mostly) From Scratch

Sanjay Madhav
University of Southern California
Los Angeles, USA
madhav@usc.edu

Matthew Whiting
University of Southern California
Los Angeles, USA
whitingm@usc.edu

Abstract

Although many introductory game programming courses increasingly use commercial game engines, we have found that this approach leads to many holes in fundamental knowledge for students. To solve this issue, we designed the current iteration of our undergraduate "Video Game Programming" course at the University of Southern California to forgo all commercial engines. Using C++, minimal starter code, and a small number of libraries, students program nine unique games during a 15-week semester. The first half of the semester has students programming 2D classics, starting with *Pong*, before switching to 3D games in the second half. The course culminates in a larger project that recreates core game mechanics of *Portal*. This work describes our course philosophy, design, and implementation details that we have refined over the past decade.

CCS Concepts

• **Applied computing** → **Education; Computer games**; • **Social and professional topics** → *Computer science education*.

Keywords

Undergraduate Education, Game Programming, C++

ACM Reference Format:

Sanjay Madhav and Matthew Whiting. 2026. No Engine Required: Teaching Game Programming (Mostly) From Scratch. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Educator's Forum (SIGGRAPH Educator's Forum '26)*, July 19–23, 2026, Los Angeles, CA, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3799829.3812528>

1 Introduction

Many introductory video game programming and game development courses use commercial engines such as Unity [Álvarez et al. 2026], Unreal [Dickson et al. 2017], or Fortnite Creative [Maraffi 2024]. These engines admittedly do have some upside: students can very quickly get game prototypes running on screen, in some cases with little-to-no coding. However, this causes many holes in fundamental knowledge. For example, students may learn how to get a ball rolling in their engine of choice but will not understand the underpinnings of time steps, delta time, or basic numeric integration.

Both authors have taught versions of introductory game programming courses with engines in the past. However, we consistently saw that the engine did so much work that the students really did not grasp the concepts. To solve this issue, about a decade ago, we redesigned our undergraduate "Video Game Programming" course to no longer use an engine and instead have students start with almost nothing. Since then, we have continued to refine our approach to the point where it is an extremely valuable experience for our students, even those with no desire to start a career in the video game industry.

2 Course Philosophy

Our course philosophy has three main pillars that we share with the students early in the semester:

- (1) Students should be able to write C++ code to solve common game programming problems as well as many game-specific problems. Unlike game engine courses, the primary focus of the course is *not* writing a game engine, but rather what the industry calls "gameplay programming" – programming various mechanics and systems that the player directly interacts with.
- (2) Students should learn to write maintainable code and not just code that they write for one assignment and never look at again. One of the main ways we achieve this is through comprehensive code reviews on each assignment.
- (3) Prepare students for game programming interviews, including learning core math concepts such as vectors, matrices, and related linear algebra concepts.

3 Course Design

Much of the course is designed around the twelve game programming assignments, and we tightly couple the progression of lecture topics with the assignments. Students have approximately one week to complete each assignment. The exception is *Portal*, which is divided into four weeks. Table 1 shows the progression of the assignments and the key topics each assignment emphasizes.

3.1 Using C++

We use C++ in the course because it is still the primary language that professional developers use, particularly at large studios. To ensure that every student can test on their own Windows, Mac, or Linux machine. We use the CMake build system [Hoffman and Martin 2011] to support these platforms. Since it is nontrivial to configure CMake to build correctly on all our target platforms (which includes a web Emscripten target), we provide the necessary CMake files in the starter code repository.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGGRAPH Educator's Forum '26*, Los Angeles, CA, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2552-4/26/07
<https://doi.org/10.1145/3799829.3812528>

Table 1: Assignments and Topics Covered

#	Game	Topics
1	<i>Pong</i>	Delta time, game loop, basic setup
2	<i>Asteroids</i>	Game object models, sprites, vectors
3	<i>Frogger</i>	Bounding boxes, level loading
4	<i>Mario</i>	Platforming physics, scrolling, animations
5	<i>Pac-Man</i>	AI state machines, path movement
6	<i>Zelda</i>	Tile mapping, A*
7	<i>Star Fox</i>	3D fundamentals, follow camera
8	<i>Mario Kart</i>	Spring camera, vehicle physics
9	<i>Portal</i> , pt. 1	First-person cameras, segment casts, quaternions
10	<i>Portal</i> , pt. 2	Portal views, teleporting
11	<i>Portal</i> , pt. 3	Actor parenting
12	<i>Portal</i> , pt. 4	User interfaces, 3D sounds

3.2 Libraries

Students write much of the code for their games from scratch. However, given that the focus of our course is on gameplay programming, there are certain concepts related to game engines that we stay away from. For these topics, we utilize open source and/or custom libraries. The primary libraries we use are:

- Simple Directmedia Layer (SDL) [Lantinga 2026] - A cross-platform library that supports creating windows, basic 2D rendering, audio, fonts, and more. Without this library, the course simply could not support multiple platforms.
- Math Library - A custom math library that includes supports for vectors, matrices, and quaternions.
- Basic 3D Renderer - For the 3D assignments, we provide a basic unlit renderer which supports texture mapping, transparency, and some features (such as stencil buffering) which are required for portal rendering.

3.3 Code Reviews

When writing code for homework assignments, students often will quickly write code and think it is done once everything works. This often leads to messy code that cannot be shown in a portfolio. It is also problematic when the students are building on the same code base every week, as the technical debt of poorly-written code can build up over fifteen weeks. To solve this problem, every assignment has a code review.

To support code reviews, we have a course style guide as well as Clang Tidy [Babati et al. 2018] and Clang Format [Clang 2026] files. Code reviews are primarily performed by our undergraduate learning assistants as part of our normal grading process. If students wish to receive an A in the course, they must respond and implement the feedback from the code reviews.

3.4 Continuous Integration

We also utilize an extensive continuous integration (CI) pipeline through GitHub Actions. Every time a student pushes code, the CI does the following:

- (1) Build the game on Windows, Mac, and Linux, reporting any build warnings and errors

- (2) Run clang-tidy for static analysis to detect common style errors such as not following the naming conventions
- (3) Run clang-format to ensure that code follows our formatting rules
- (4) For certain assignments, run unit tests to validate certain features (such as AABB intersection)

4 Results

Although we acknowledge that student evaluations should be taken with a grain of salt, by and large the evaluations are extremely positive, as these comments demonstrate:

- "I liked that this course really allows for the exploration of fundamental ideas that make up games and game engines, while still staying big picture enough to get an understanding of related fundamental mathematical and theoretical concepts as well."
- "I think the labs genuinely incorporate the class content really naturally and were by far the most helpful parts of actually learning and understanding what was covered"
- "While I would definitely say I was challenged in this course, I also felt rewarded whenever my homework was complete. What I am learning feels very applicable to anything I may endeavor to do in the future."

The feedback from course alumni has also been very positive. Dozens of former students now work for major video game developers. Anecdotally, when we run into old students at conferences, we are told how our "Video Game Programming" course was one of the most important for launching their career in video games.

5 Conclusion and Future Work

Using commercial game engines for education is a shortcut to quickly getting a game up and running. However, this comes at the cost of a decreased understanding of the fundamental concepts. Instead of using a commercial engine, our "Video Game Programming" course has students build out a series of increasingly complex games over the course of the semester. This leads to a greater understanding of the core concepts that are required for further study in the field.

As instructors, we are also satisfied with the current state of the course. However, with AI coding tools becoming increasingly more powerful, it is likely that we will have to reinvent the course once more to account for the massive paradigm shift we are witnessing.

Acknowledgments

We would like to thank our colleagues at the University of Southern California, our undergraduate learning assistants who do much of the grading and code reviews, as well as all the students who have taken our courses over the years.

References

- Villagómez-Palacios Álvaro, Cristian Vidal-Silva, et al. 2026. Videogame Programming & Education: Enhancing Programming Skills Through Unity Visual Scripting. *Computers* 15, 1 (2026), 68.
- Bence Babati, Gábor Horváth, Viktor Májer, and Norbert Pataki. 2018. Static Analysis Toolset with Clang. In *Proceedings of the 10th International Conference on Applied Informatics*. 23–29. doi:10.14794/ICAL.10.2017.23
- Clang. 2026. *ClangFormat – Clang 23.0.0git Documentation*. Retrieved February 9, 2026 from <https://clang.llvm.org/docs/ClangFormat.html>
- Paul E. Dickson, Jeremy E. Block, Gina N. Echevarria, and Kristina C. Keenan. 2017. An Experience-based Comparison of Unity and Unreal for a Stand-alone 3D Game Development Course. In *Proceedings of the 2017 ACM Conference on Innovation and Technology in Computer Science Education* (Bologna, Italy) (ITiCSE '17). Association for Computing Machinery, New York, NY, USA, 70–75. doi:10.1145/3059009.3059013
- Bill Hoffman and Kenneth Martin. 2011. CMake. In *The Architecture of Open Source Applications*, Brown. Amy and Greg Wilson (Eds.). Lulu.com, Chapter 5, 67–77.
- Sam Lantinga. 2026. *Simple DirectMedia Layer - Homepage*. Retrieved February 9, 2026 from <https://libsdl.org/>
- Topher Maraffi. 2024. Level-Up Logics: Leveraging Three Game Design Platforms to Teach Coding. In *ACM SIGGRAPH 2024 Educator's Forum* (Denver, CO, USA) (SIGGRAPH '24). Association for Computing Machinery, New York, NY, USA, Article 11, 2 pages. doi:10.1145/3641235.3664434

A Additional Materials

For further information on the undergraduate "Video Game Programming" course at the University of Southern California, our course website has videos, assignments, and detailed information about the overall course structure: <https://itp380.org/>.